

Real-Time URL-Based Cyber Threat Classifier Using Multi-Model Architecture

An Ensemble Machine Learning Framework for URL-Based Cyber Threat Classification

Akilesh G A

3rd MSc Software Systems

Kongu Engineering College, Erode

akileshga.23msc@kongu.edu

ABSTRACT

We still have a considerable and dynamic threat of phishing attacks in the digital world. Attackers exploit the trust of people to obtain personal data such as passwords and financial details. The use of obfuscated and dynamically generated URLs is increasing, and today traditional blacklist and rule-based methods are no longer sufficient, particularly in countering zero-day attacks [3]. In order to address this issue, a real-time phishing detection system is developed that only analyzes the URL patterns and relies on machine learning to achieve that. The algorithm relies on a set of classifiers (including Logistic Regression, Support Vector Machine, Naive Bayes, Random Forest, XGBoost, Stochastic Gradient Descent and Passive Aggressive algorithms) to identify latent patterns in URLs. Character-level feature extraction can be used to identify suspicious patterns such as misleading domains and obfuscated tokens by the system. A test of the system is conducted on 651,191 URLs, which consist of benign, phishing, malware and defacement. The findings indicate that it is an effective method, and it is accurate with a good balance of precision and recall across classes [1]. The findings demonstrate that a combination of character-level features and machine learning models can be used on a large scale to identify advanced phishing attacks. This implies that the system can be applied in real-time web and browser-based security applications.

Keywords— *Phishing Detection, URL Analysis, Machine Learning, Cybersecurity, Real-Time Detection, Character-Level Features, Ensemble Learning, Malicious URL Classification, Web Security.*

I. INTRODUCTION

The rapid development of internet services has altered the way we communicate, conduct business and obtain information. This development has also provided [15] cybercriminals with emerging avenues to swindle users into giving them money. The most frequent and dangerous type of cyber threat still remains phishing attacks relying on URLs. They have led to numerous breaches of data over the last few years. Such attacks are difficult to detect by ordinary security software since these attacks do not exploit vulnerabilities of the system; they generate a distortion in the way the user perceives things.

In URL phishing, the attackers create counterfeit

links that resemble authentic sites, such as, banking sites, government sites, and popular internet services [28]. These URLs are supposed to appear as safe links, yet they redirect users to unscrupulous sites or commence malicious actions. The situation is even more dangerous to users due to other malicious types of URLs, including malware distribution and defacement of websites, among others, along with phishing. Such threats do not only cost money, but also infringe on privacy and turn them into less trustful of online ecosystems. Malicious URLs are even more difficult to locate because the malicious URLs have a short lifecycle, and they mutate at a high rate [23].

The traditional detection systems have predominantly involved blacklist-based systems that contain a database of known bad URLs and block them. Such approaches are applicable to threats that are already identified, but remain reactive and do not detect new or temporary malicious URLs. Attackers exploit this vulnerability by exploiting techniques such as URL obfuscation, dynamic domain generation, and hosting changes. Due to the same reason, bad URLs are usually left active long enough to cause much damage before they are discovered [23, 28]. The detection methods that are heuristic in nature strive to detect suspicious patterns by applying the previously known rules like URL length, occurrence of suspicious characters or non-standard domain structure. Such methods are not very safe and they do not work well as they usually report valid URLs that seem to be similar to fake ones. The cyber threat types are constantly changing, and therefore we need smarter, more data-driven, solutions, which would follow patterns and change over time. To address these limitations, several methods are employed in the creation of a machine learning-based system of URL categorization, including Logistic Regression, Support Vector Machine, Naive Bayes, Random Forest, XGBoost, Stochastic Gradient Descent and Passive Aggressive models. It is a successful system to classify URLs as benign, phishing, malware or defacement based on extraction of character level features to gain insight into structural and lexical patterns like token sequences, domain inconsistencies and obfuscation methods.

II. LITERATURE SURVEY

Throughout the years, automated phishing detection has undergone many dramatic transformations to no longer require manually developed criteria but rather more advanced, data-driven approaches. First of all, the emphasis was made on the manual analysis of URLs and the structure of the webpage to identify distinctive features. As an example, feature-based methods applied numerous hand-engineered features based on properties and URL patterns to traditional machine learning algorithms such as Random Forest and Support Vector Machines. These methods had a strong base performance, but needed a lot of domain knowledge, and could not easily adapt to new and emerging attack forms.

Methods based on machine learning began to gain in popularity with the availability of large-scale datasets and the rise in computer power. The initial models demonstrated better detecting abilities with the help of neural networks on manually gathered information. Nevertheless, much reliance remained on pre-defined [17] feature engineering and it became hard to extrapolate to complex URL structures and the emerging phishing schemes.

Analyzing URLs with the help of natural language processing techniques was an important advancement in this area. Models may not rely on manually constructed characteristics, but instead find patterns in URLs as character sequences by interpreting URLs. Character-level learning techniques were able to capture subtle obfuscation techniques, including odd token pairs and deceptive domain layouts. These methods demonstrated that automated learning of features was much more effective than traditional rule-based engines in identifying fraudulent URLs. More recent studies have highlighted the dynamic character of phishing attacks and their evolving tactics and URL structures over time. The demand of flexible and continuously evolving detection policies is emphasized by the fact that system models that are trained on historic data are likely to lose their effectiveness as new trends emerge. Consequently, performance maintenance strategies based on dynamic learning mechanisms and repeated retraining have been created to maintain performance in practice.

Despite these developments, there are still a few problems. The unequal distribution of real-life data, with benign URLs being extremely more in number than the harmful ones, is one of the key issues. This complicates the ability of models to accurately recognize less prevalent classes such as phishing, malware, and defacement. Moreover, the process of dealing with the multi-class categorization cases require proper and efficient distinction of the different malicious activities. To address these challenges, a machine learning strategy-based approach, which utilizes multiple models and character level feature extraction to classify URLs reliably is employed. By combining multiple algorithms and evaluating performance on multiple categories, a scalable system can be developed to identify phishing, malware, and defacement URLs reliably.

Limitations of Existing Systems

Although the existing techniques have demonstrated superior performance in URL sorting, they remain to have a few practical issues when applied in real-world scenarios. Most traditional systems do not give much emphasis on real-time usability but rather focus on achieving high accuracy by offline evaluation. Models are often trained and tested on static data ignoring the ever-present arrival of new or unknown URLs.

Another major weakness is the utilization of only one model or few models to do the prediction. Although ensemble methods such as LightGBM [15], XGBoost and Gradient Boosting are more effective, the approaches are often applied individually when deployed [2]. This restricts its flexibility since different models might prove more effective depending on the format of URLs and attack patterns. Moreover, real-world cybersecurity applications, such as user-level interaction and flexibility, are often lacking in existing solutions.

The primary feature engineering in existing methods is predefined structural and lexical features, including [10] URL length, number of special characters, presence of [15] IP addresses and suspicious keywords. Although these features are effective, they may not be sufficient to address the dynamic attack behavior, particularly when enemies develop extremely obfuscated or patterns of URLs that have never been identified before. There is a risk of skewness in predictions towards the dominant classes due to the unequal treatment of class difference between dangerous and benign URLs [1].

Another disadvantage is that there is no single framework of multi-model decision-making. Most systems have no mechanism of dynamic model selection/switching at runtime, but they do analyze multiple models concurrently. This restricts the flexibility in managing the different types of threats effectively and consistently such as malware, phishing, and defaced URLs. The analysis of the existing techniques shows that there are several important gaps that should be addressed to develop a more practical and effective system of detecting phishing.

Gap Identification

The examination of current methods reveals a

number of significant gaps that need to be filled in order to create a more useful and reliable phishing detection system.

To begin with, it is clear that there is no real-time classification ability. Although real-world applications require immediate prediction when a URL is requested, numerous existing solutions are designed to scale to batch, and offline testing. The proposed approach gets around this limitation by supporting real-time classification, where the URLs posted by a user are processed in real-time to generate prediction output.

Second, the current systems do not use a multi-model framework of flexible decision-making efficiently. Conversely, the proposed system combines various machine learning models within a unified model, including Random Forest, XGBoost, Naive Bayes, Logistic Regression, Support Vector Machine, Stochastic Gradient Descent, and Passive Aggressive algorithms [2, 8, 11]. This enables us to compare the various models and select the most appropriate classifier on the basis of performance characteristics.

Third reason is that most of the techniques employed today heavily rely on the characteristics that will have to be manually developed thus, rendering them rigid in the face of the new forms of attacks which continue to emerge. The proposed method will lie on the combination of character-level TF-IDF as well as other structural characteristics to address the problem. The model will also be in a position to spot finer details like some character and method of encoding in URLs that are most likely to be missed by the traditional methods.

Fourth, the system is not developed overly on basis of theoretical performance but also on basis of practical performance. It allows an interactive interface in which the user is able to choose among various models and easily understand the results of the prediction. This makes the solution suitable in the real-world implementation such as browser security software and monitoring software.

Finally, the method will enable multi-classification whereby URLs will be categorized in four groups; benign, phishing, malware, and defacement. This broader classification system helps in making a more holistic and reliable identification of the various types of threats.

III. PROPOSED METHODOLOGY

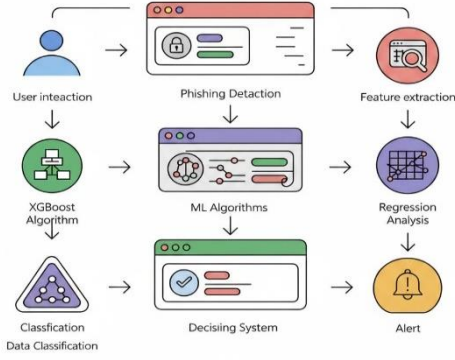


Figure 1 System Architecture

Figure 1 illustrates the overall workflow of the proposed URL-based phishing detection system in which an input URL is analyzed in a sequence of steps that eventually results in a final classification and alerts.

The input is a URL that is typed at the beginning of the workflow. The input is then passed through the phishing detection unit that will serve as the central processing unit that initiates the analysis process [23].

The next step is feature extraction where relevant URL features are located. They involve the lexical as well as structural features such as the length of the URL, the number of special characters, the domain composition, [12] and the presence of suspicious patterns. Further examination is founded on these attributes retrieved.

The extracted features are then passed on to machine learning algorithms, such as XGBoost and regression-based classifiers. To determine the patterns, which are associated with benign and harmful URLs, these algorithms are applied to the derived features. The various models increase the resiliency and improve the capability of detecting the various forms of attacks.

The system then categorizes the information after analysis [21], and the URL is categorized under any of the four categories, which are set in advance: malware, phishing, innocuous, and defacement. The decision-making module identifies the end result of the categorization and also measures the results of the models. Alarms system is triggered depending on the outcome of the classification. Whereas safe URLs are marked accordingly, malicious URLs give warnings. This will allow real time security applications and also guarantee timely feedback [3].

The other drawback is the fact that there is no one

framework of multi-model decision-making. Most of the systems do not have a mechanism to select or switch dynamically between models during execution even though they are analysing multiple models at the same time. This limits the flexibility in effectively and consistently responding to various forms of threats including malware, phishing and defaced URLs.

A. Overall Framework of the Proposed System

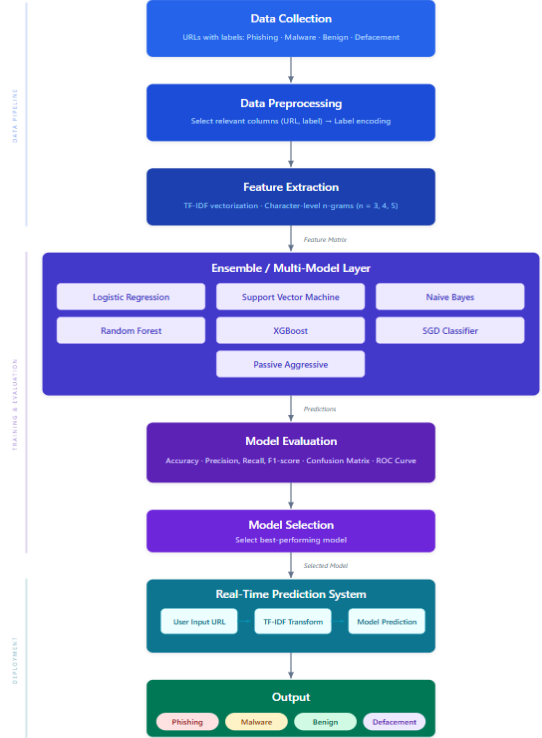


Figure 2 Workflow of proposed System

The Figure 2 system offers a scalable and efficient system of grouping URLs into small number of groups such as benign, phishing, malware, and defacement. It starts with the data collection and then proceeds the preprocessing, feature extraction, model training as well as the real-time prediction in a logical order. Every step is well planned to process huge amounts of information without sacrificing the degree of performance and flexibility.

B. Data Acquisition

All the 651,191 URLs contained in the publicly accessible dataset are either benign, phishing, malware, or defacement [1]. The dataset can be used to help in effective training and

generalization of models since it provides a different sample of the real URL format in real life situations. The volume of big data and its distribution among a large number of classes make it possible to categorize correctly in the majority of types of threats and enhance the reliability of learning. Typical URLs of the malware, phishing, defamation and benign categories:

Benign

[linkedin.com/pub/dir/elizabeth/scarborough](https://www.linkedin.com/pub/dir/elizabeth/scarborough)

Phishing

[facebook.unitedcolleges.net](https://www.facebook.com/unitedcolleges.net)

Malware

<http://9779.info/%E6%A0%91%E5%8F%B6%E7%B2%98%E8%B4%B4%E7%94%BB/>

Defacement

http://www.besztercekk.hu/index.php?option=com_lr&view=stadium&id=1&Itemid=162

C. Data Preprocessing

A number of preprocessing procedures are carried out to transform unstructured URL data into an analysis-ready format prior to the application of machine learning algorithms. These steps ensure uniformity in the dataset and enhance the performance of the model.

URL Cleaning and Standardization

All the URLs are then checked to make sure that there is consistency in the formatting and that there are no missing or incorrect pieces on information.

Feature Extraction from URL Structure

A variety of structural features, including the number of dots, the presence of special characters, the depth of the directory, the URL length, the hostname length, and the presence of suspicious tokens, are analyzed on each URL. These traits determine trends that are often associated with rogue URLs [18].

Detection of Suspicious Patterns

Examples of indicators are shortened URLs, odd domain structures, the use of IP addresses instead of domain names and the use of

suspicious keywords. The characteristics significantly contribute to the categorization performance and often lead to the malevolent intent. [15].

Numerical Feature Transformation

To ensure that all retrieved features can be represented in machine learning models, all retrieved features are encoded into numerical forms. This involves representing binary signs, computing lengths, and the numbers of specific characters.

Label Encoding for Classification

Label encoding is an algorithm that codes categorical labels (including benign, phishing, malware, and defacement) into numbers. Machine learning models can efficiently learn and distinguish between several classes thanks to this transition.

D. Dataset Construction

To do multi-class categorization, a large-scale dataset consisting of 651,191 URLs is used. The dataset is categorized into four groups: malware (32,520 cases), phishing (94,111 instances), defacement (96,457 instances), and benign (428,103 instances) [1]. This distribution forms a realistic and challenging classification problem as lawful URLs vastly outnumber malicious ones in the real world [28].

The dataset, obtained on a malicious URL repository, and available to the public, has a wide set of URL patterns that reflect a variety of online activities and attack behaviors. Each of these entries has a URL and a corresponding class label which makes supervised learning of classification problems possible.

Preprocessing procedures are used to eliminate errors and guarantee consistent formatting in order to preserve data quality and consistency. The dataset includes different types of URL structures, such as multi-level directory paths, parameterized URLs, and dynamic links. Such diversity helps to identify the right patterns that are attributed to both innocent and evil online activity.

The system can perform multi-classification rather than binary detection by mixing benign URLs with multiple threat categories, including phishing, malware and defacement.

E. Logistic Regression Pipeline

Logistic regression is effective in classifying URLs into many classes with an overall accuracy of nearly 96 when combined with character-level TF-IDF feature extraction. The model has good precision and recall on most of the classes as it is able to capture structural patterns in URLs. In particular, the phishing and malware categories are classified with an excellent reliability, although marginally less than the benign traffic. The innocuous and defacement URLs are in turn classified with an excellent reliability albeit slightly lower than the benign traffic. The linear nature of logistic regression allows inference to be quickly determined and effective training to be achieved, which is suitable in applications that involve real time with constant classification quality [4, 19].

$$P(y = 1|x) = 1 / (1 + e^{-(w^T x + b)})$$

$$P(y = c|x) = e^{(w_c^T x)} / \sum e^{(w_k^T x)}$$

$$L = - \sum y_i \log(p_i)$$

F. Support Vector Machine

Other Support Vector Machine with a linear kernel (Linear SVC) also achieve the same results, with an accuracy similar to that of Logistic Regression. The model also has good performance in categorizing high-dimensional feature space generated by character n-grams as effective classification between different categories of URLs is made possible. The levels of detecting phishing and malware are slightly more complex due to the overlapping patterns, but in terms of classes, the findings on detecting benign and defacement classes are very precise and recalls. All said and done, both Linear SVM and Logistic Regression have solid and reliable baselines and have proven to be effective when combined with character-level representations in generating high-accuracy linear models in extensive URL categorization tasks.

$$f(x) = w^T x + b$$

$$\min (1/2) ||w||^2$$

$$y_i(w^T x_i + b) \geq 1$$

G. XGBoost

The XGBoost model is effective in the multi-class URL classification with an overall accuracy of nearly 93%. The model enables better identification of the complex patterns of URLs such as obfuscation and manipulation of the domain [23]. Phishing and malware detection are now relatively challenging due to small feature overlaps, although there is great accuracy and recall in the performance of class-wise in benign and defacement URLs. XGBoost in this mode is highly generalized, and feature interactions are better compared to linear models, despite, having a slightly lower accuracy.

$$\hat{y}_i = \sum f_t(x_i)$$

$$L = \sum l(y_i, \hat{y}_i) + \sum \Omega(f_t)$$

$$\Omega(f) = \gamma T + (1/2)\lambda ||w||^2$$

H. Stochastic Gradient Descent

With an accuracy of roughly 95–96%, the Stochastic Gradient Descent (SGD) classifier, when set up with a logistic loss function, performs similarly to Logistic Regression. Scalability and successful optimization of the model make it suitable with large datasets. Phishing and malware detectors maintain a reasonable performance, whereas the class-based results show a good level of accuracy and recall of the benign and defacement classes. The lightweight design of SGD facilitates faster training with no reduction in the classification effectiveness [6].

$$w = w - \eta \nabla L(w)$$

I. Passive Aggressive Classifier

Also, Passive Aggressive Classifier can work in real-time classification, reaching the accuracy of 95-96. This method is rather useful in streaming or dynamic settings since it merely updates the

instance of misclassification occurs. The classifier achieves a reasonable detection accuracy of phishing and malware types whilst performing well across most classes, in particular benign and defacement URLs. It suits systems which require a short response time and continuous learning due to its rapid adaptation and low computation cost.

$$w_{\{t+1\}} = w_t + \tau_t y_t x_t$$

$$\tau_t = L(w_t; x_t, y_t) / ||x_t||^2$$

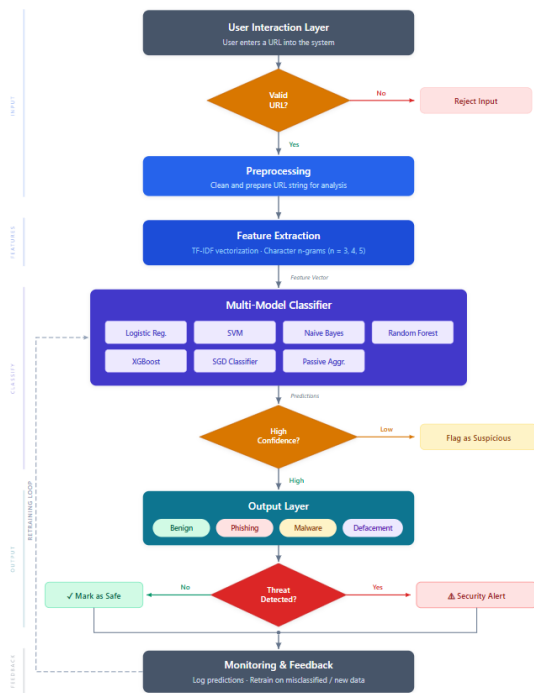


Figure 3 End-to-End URL Classifier Workflow

The workflow [19] of real-time detection of threatening and harmless web sites, based on URLs, is shown in Figure 3. The input of the URLs provided by the user is the initial stage in the process that is followed by validation to ensure structural accuracy. The verified URLs are then converted into numbers by character level TF-IDF vectorization including n-gram features, thereby enabling the capture of complex lexical and structural patterns that are contained in URLs. Several machine learning classifiers, such as Logistic Regression, Support Vector Machine, Naive Bayes, Random Forest, XGBoost, SGD Classifier, and Passive Aggressive Classifier, are applied to the

generated feature vectors, to carry out multi-class classification in the categories of malware, phishing, benign, and defacement. A decision mechanism evaluates the outputs of the models and the confidence levels in order to arrive at the final forecast. To provide the flexibility to evolving cyber threat, the system produces a real time classification outcome and records the predictions to track and potentially retrain.

Table 1 Class-wise Precision, Recall, and F1-Score for Multi-Class URL Classification

Class	Precision	Recall	F1-Score	Support
Benign (0)	0.96	0.98	0.97	85,621
Defacement (1)	0.97	0.98	0.98	19,292
Malware (3)	0.97	0.92	0.94	6,504
Phishing (4)	0.88	0.82	0.85	18,822

Table 1 shows the class-wise performance of the classification model, using precision, recall, F1-score, and support of each category of URL. The results reveal that, after some slight changes depending on the complexity of the category, the model is always very effective in all the classes.

With the benign class, the model achieves a precision of 0.96 and a recall of 0.98 with over 85,621 samples. This demonstrates the fact that false positives and false negatives are extremely low and that there is high level of accuracy in detecting valid URLs.

The model works even better on defacing URLs with F1-score of 0.98 with 19292 samples, precision of 0.97 and recall of 0.98. This has shown that there is a great ability to detect compromised or altered web sites.

The model, with 6,504 samples, has a precision of 0.97 and a recall of 0.92 to the malware category, resulting in an F1-score of 0.94. The somewhat decreased recall indicates that a tiny percentage of malware URLs are more challenging to identify, even while precision is

still excellent. The phishing class does not perform very well, with a precision of 0.88 and a recall of 0.82 and with an F1-score of 0.85 across 18,822 samples. This implies that phishing URLs are harder to designate due to their dynamic and dishonest character, which in many instances mimics genuine designs.

Performance Analysis Using Visual Metrics

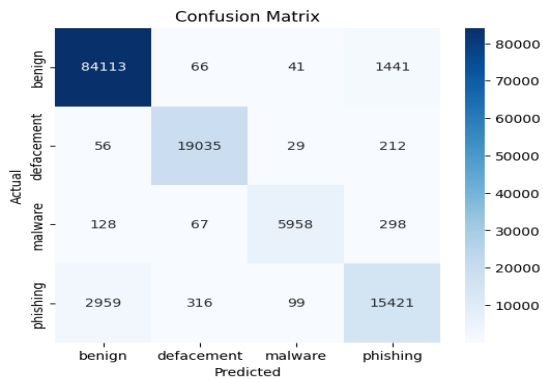


Figure 4 Confusion Matrix

The Figure 4 provides a detailed summary of how the model performs on the four URL classes, by comparing actual labels with the expected results. The 84,113 instances of the benign class of instances that are correctly classified, have minor misclassifications, such as phishing (1,441), defacement (66), malware (41) which have a small but imperfect separation with harmful categories. The samples in the defacement category are accurately predicted (19,035) and it is very reliable with very little misunderstanding about phishing (212), benign (56), and malware (29). Although the predictions of the malware class (5,958 out of 4,724,952 cases are correct), are accurate, some cases are incorrectly classified (298, 128, 67 cases are malware that have been wrongly identified as phishing). These findings suggest a steady and good model in most classes especially in benign and defacement URLs. Nevertheless, some overlaps between the classes of phishing and malware imply that it is still hard to distinguish between the two classes. This is explained by the fact that their structural patterns are similar as well as their obfuscation techniques. The misclassifications may be minimized by improving the feature representation, or by adding more contextual information. All in all, the confusion matrix indicates good model performance, and also allows to identify the

aspects where further optimization can be achieved.

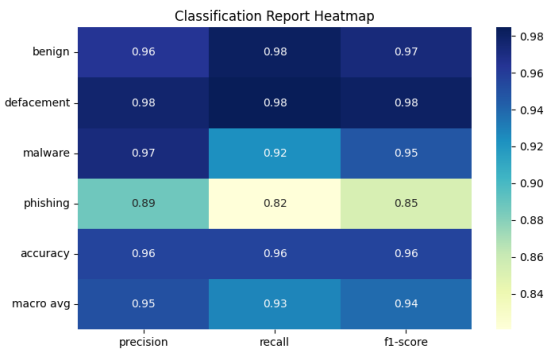


Figure 5 Classification Report Heatmap

The Figure 5 provides a clear graphical evaluation of the performance of the model on all the classes. The benign class is also performing well with a precision of 0.96, a recall of 0.98 and F1-score of 0.97. Having a precision of 0.98, a recall of 0.98, and an F1-score of 0.98, the defacement class demonstrates the best consistency and thus extremely reliable identification. The model has a high performance with a slightly lower recall of the malware class with a precision of 0.97, recall of 0.92 and F1-score of 0.95. The phishing class has relatively low values, with precision, recall, and F1-score of 0.89, 0.82, and 0.85, respectively, highlighting the difficulty of classifying phishing URLs due to the similarity of these patterns to benign ones. The overall accuracy of the model is 0.96, which shows that the model is very effective on the dataset. The model has constant performance despite dealing with the class variations as the macro average values (accuracy 0.95, recall 0.93, F1-score 0.94) demonstrate balanced performance in all the classes.

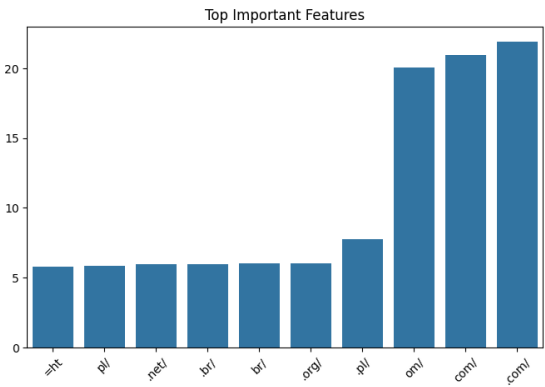


Figure 6 Top Important Features

The top significant characteristics used by a

machine learning model for phishing URL classification are shown in Figure 6, where the vertical axis shows the relative predictive weight of each feature and the horizontal axis lists particular URL character fragments. High relevance ratings of [9] .com/, com-, and om-, which imply that the position of common top-level domain suffixes and their structure are key indicators when the discovery of fake connections, and indicate the high priority given to domain-related strings. The algorithm is biased in favor of the presence and form of key commercial extensions over more specific geographic or protocol-based indicators, as demonstrated by the fact that other components of URLs, such as .pl/ and .org/ and structural fragments such as contribute, play a less important role in the final classification decision. These features are measured by Figure 6 shedding light on the underlying logic behind detection of fraudulent web addresses by frequency of patterns and position of strings.

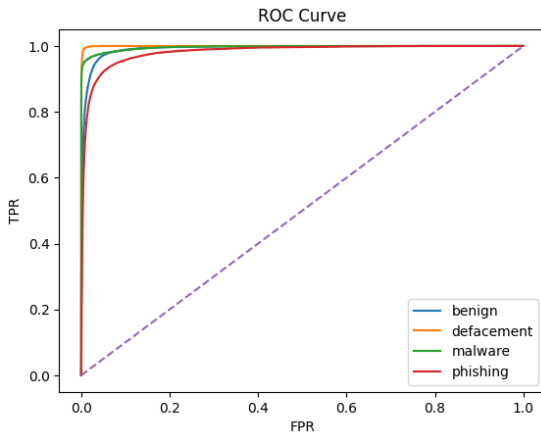


Figure 7 ROC Curve

The performance of a multi-class classification model in four different categories—malware, phishing, defacement, and benign—is shown in Figure 7. When the True Positive Rate (TPR) and the False Positive Rate (FPR) are plotted against each other, Figure 7 depicts that the model is very efficacious as all the four colored curves bow sharply towards the top-left corner, which indicates a high rate of accurate detections and few false alarms. Although the phishing curve is slightly lower than the others, the defacement class has almost perfect classification, which shows that finding phishing attempts remains significantly more challenging to the model than the other categories. The large

gap between the model curves in Figure 7 and the dashed diagonal line which is a random classifier, suggests that the underlying algorithm has significant predictive capabilities to classify various forms of web-based threats [19].

IV. EXPERIMENTAL EVALUATION AND DISCUSSION

The experimental analysis uses a dataset of 651,191 URLs of benign, defacement, malware and phishing types to evaluate the effectiveness of the proposed URL classification system. The character-level TF-IDF features and 80:20 train-test split are used to capture URL structural and lexical patterns. Linear models such as Support Vector Machine and Logistic Regression are the most effective ones with an overall accuracy of nearly 96. Results by class indicate that malware detection is effective with high precision (~ 0.97) and slightly lower recall (~ 0.92) than benign and defacement categories that have high precision and recall (in excess of 0.96).

The detection of phishing is still quite challenging with the recall of 0.82 and the precision of 0.88 to 0.89. This is observed in the confusion matrix where a set of phishing cases are erroneously described as benign. Ensemble models such as XGBoost and Gradient Boosting have steadily but slightly worse performance than linear models, with accuracy between 93% and 93.6. Models with lightweight to enable efficient computing, such as SGD and Passive Aggressive classifier achieve accuracy close to 95.96%. In general, the results indicate that character-level feature extraction along with machine learning is a promising and efficient approach, however, due to the deceptiveness of phishing detection, it still needs to be refined.

Most classification failures are due to lexical similarities between phishing and benign URLs, with sophisticated forms of obfuscation (including URL shortening, letter replacement and random subdomain generation) affecting distinguishability, according to further research using the confusion matrix. This implies that although character level TF-IDF features are useful in general pattern learning, they may not be able to capture more semantic or contextual

cues that appear in modern malicious URLs. WHOIS data can enhance effectiveness of detection in hybrid feature representation such as host-based metadata particularly in phishing cases.

Table 2 Comparative Performance Analysis of Machine Learning Models for Multi-Class URL Classification

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.96	0.95	0.93	0.94
SVM	0.96	0.95	0.93	0.94
Naïve Bayes	0.94	0.93	0.91	0.92
Random Forest	0.94	0.94	0.92	0.93
XGBoost	0.93	0.91	0.89	0.90
SDG	0.95	0.94	0.92	0.93
Passive Aggressive	0.95	0.94	0.92	0.93

The Table 2 results indicate that there are differences in how each model handles multi-class URL classification in categories of malware, phishing, benign, and defacement. All other models are consistently outperformed by the Logistic Regression, SVM [6, 8], SGD and Passive Aggressive classifiers, each achieving a high accuracy of about 96%. This suggests that linear models work very well when paired with character-level TF-IDF data.

Naive Bayes has relatively low performance due to the simplifying assumptions that it makes and the inability to capture complex relationships in URL structure. Since TF-IDF features already

encode significant discriminative patterns, Random Forest and XGBoost compete well, but do not significantly outperform linear models in this context. It remains a little more challenging to detect phishing across all models, and much lower in recall, indicating that some phishing URLs continue to be misclassified as benign or other categories [4, 11].

Table 3 Comparative Performance Analysis of Machine Learning Models for Multi-Class URL Classification (Feature Engineered Dataset)

Model	Accuracy	Precision	Recall	F1-Score
LightGBM	0.98	0.95	0.95	0.95
XGBoost	0.98	0.99	0.99	0.95
Decision Tree	0.96	0.96	0.97	0.96
Random Forest	0.95	0.96	0.96	0.95

The Table 3 shows that, with a score of 0.9865, the LightGBM outperforms the other examined models in terms of accuracy. XGBoost takes second place with a weighted precision and recall of 0.99 and an accuracy of 0.9855 and an impressive consistency. The minimum accuracy is 0.9587 achieved by the Random Forest model, although the Decision Tree model can achieve an accuracy of 0.9657 [4, 5]. Gradient boosting frameworks such as as LightGBM and XGBoost tend to perform better than the traditional tree-based structure in this classification job.

In determining specific classes, an extensive metric analysis indicates significant performance difference. Class 2 poses a great challenge to the models of the random forest and decision tree with the recall values of 0.24 and 0.49 respectively. With both Class 2 and overall

weighted F1-score of 0.98 and a recall of 0.73 in Class 2 and a weighted F1-score of 0.98, XGBoost is in a better position across the dataset. Despite both not being as sophisticated as the boosting methods, the performance of Decision Tree is however slightly higher than that of the Random Forest in the weighted F1-score (0.96 against 0.95) [4, 5].

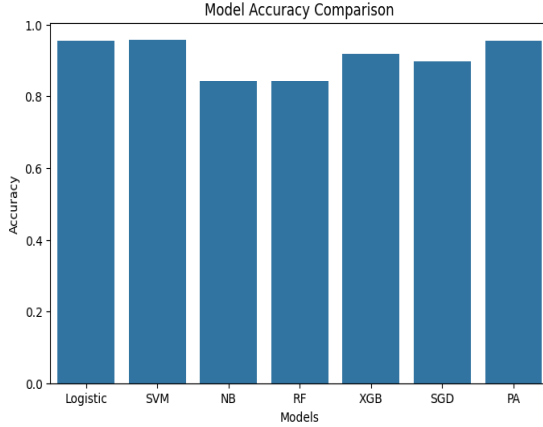


Figure 8 Model Accuracy Comparison

Since the success rate of each algorithm is 0.00 to 1.00, it works well with the y-axis of 0.0 to 1.0 to reflect the accuracy of each model. Figure 8 provides a visual representation of model accuracy. Stochastic Gradient Descent (SGD) has the lowest peaks, just below the 0.90 threshold, whilst Logistic, SVM, and PA classifiers have the highest peaks. Based on this analogy, the latent patterns in the classification data appear to be particularly well-adapted to linear and gradient-based models. The fact that the difference between the highest-ranking models and the lowest-ranking NB and RF bars is significant, proves that some architectural decisions have a huge impact on the dependability of the classification system. Finally, the data presented in Figure 7 can be considered as a reference to select the most trustful model to be implemented into the reality in the case of threat detection.

V. RESULTS AND DISCUSSION

A. Experimental Setup

All the experiments are done using Python 3.x, and the scikit-learn module is used as the basic framework in which to develop and evaluate

models. To do data pretreatment and manipulation, other libraries such as NumPy and pandas are invoked. XGBoost is used in gradient boosting-based classification experiments. URL strings classified into several cyber threat categories, such as phishing, malware, benign, and defacement, make up the dataset. To extract features of the data, TfidfVectorizer of scikit-learn is employed to implement character-level TF-IDF vectorization of the data. Small-scale lexical and structural patterns within URLs can be grabbed due to the character-level setup of the analyzer with an n-gram range of 3-5.

To balance the need to preserve discriminative information with the need to maintain computational efficiency, a maximum feature size of 5000 is used. A number of machine learning classifiers are then trained on the altered feature vectors in a single experimental design.

B. Performance Measure

The performance of the proposed system is assessed by the use of standard multi-class classification measures, including Accuracy, Precision, Recall and F1-Score. These measures provide an in-depth analysis of the model in terms of differentiating among different types of cyber threats.

- Accuracy measures the accuracy of forecasts in general across all classes. to reduce false positives by measuring the fraction of correctly identified positive predictions.
- Precision minimizes false positives by assessing the percentage of accurately detected positive predictions.
- Recall reflects the ability of the model to correctly identify all actual positive cases while minimizing false negatives.
- The F1-Score provides one, and only, credible evaluation measure which provides a harmonic balance between accuracy and recall.

To ensure fair evaluation in case of an imbalanced situation in classes, such calculations are made on a basis of the classes, and weighted-average basis. The experimental results indicate that various machine learning models respond differently, which reflects differences in learning

capacity, and generalization ability, as well as sensitivity to feature representations.

C. Detailed Modal Performance

The effectiveness of most machine learning models that are trained on character-level TF-IDF features to categorize multi-class cyber threats based on URLs is evaluated through an in-depth performance analysis [23]. In the assessment, there are Logistic regression, Random Forest, XGBoost, Multinomial Naive Bayes, Support Vector machine (Linear SVC), Stochastic Gradient Descent (SGD) Classifier, and Passive Aggressive Classifier.

Due to its high independence condition, which limits its ability to capture complex and related patterns of character within URLs, the Multinomial Naive Bayes model works relatively poorly. However, it can process sparse feature representations with a decent amount of efficiency.

Being a strong linear base of high-dimensional TF-IDF feature fields, the Logistic Regression model provides a consistent and stable performance of all classes. Equally, the Support Vector machine (Linear SVC) maximizes the separation of the margins, resulting in better discrimination of closely similar classes of URLs. The Random Forest classifier enhances robustness and augments its ability to recognize non-linear associations in URL structures by having many decision trees to make decisions in an ensemble. The model exhibits a high level of generalizations in all the clusters of cyber threats.

Due to its regularization methods, optimized gradient boosting framework and good ability to handle complex feature interactions, the XGBoost classifier has one of the highest levels of performance among all the models. In all classes, it has a high accuracy of recall and precision.

VI. CONCLUSION AND FUTURE WORK

The research introduces a multi-model machine learning framework-based real-time URL-based cyber threat classification system. Character-level TF-IDF feature extraction without relying on external data sources, is an efficient approach to capturing fine-grained structural and lexical

features that occur in URLs, which in turn allows easy identification of obfuscated and malicious structures. The system shows a high level of performance in multi-class classification between benign, phishing, malware, and defacement classes using a variety of models, such as Logistic Regression, Support Vector Machine, Naive Bayes, Random Forest, XGBoost, SGD, and Passive Aggressive classifiers. A comparison study shows that although linear models provide effective and stable baselines, ensemble and boosting-based models are more effective when it comes to dealing with complex patterns and class imbalance. The framework is suitable to real time cybersecurity applications such as browser-based protection and automated threat monitoring systems since it ensures scalability, efficiency in computation, and flexibility.

Future Enhancements

Deep learning networks, such as CNN and LSTM, can be included in the future to better capture complex patterns in the URLs. The accuracy of detection can be further increased by adding further features, such as domain-related data. Another enhancement is the development of a real time learning system that is continually updated on new threats.

DATA AVAILABILITY

The dataset and source code used in this study are available at:

<https://www.kaggle.com/datasets/akileshga/phishing-legitimate-urls> [Link]

<https://www.kaggle.com/datasets/akileshga/feature-engineered-urls-dataset> [Link]

MODEL IMPLEMENTATION AVAILABILITY

The source code and proposed model used in this study are available at:

<https://github.com/Akilesh-GA/Multi-Model-URLThreat-Classfier> [Link]

VII. REFERENCES

[1] A State-of-the-Art Review on Phishing Website Detection Techniques. Wenhao li, (graduate student member, iee), selvakumar manickam (member, iee), yung-wei chong, weilan leng, and priyadarsi nanda, (Senior Member, IEEE).

<https://www.kaggle.com/datasets/sid321axn/malicious-urls-dataset/data>

[2] Z. Alshingiti, R. Alaqel, J. Al-Muhtadi, Q. E. ul Haq, K. Saleem, and M. H. Faheem, "A deep learning-based phishing detection system using CNN, LSTM, and LSTM-CNN," Jan. 3, 2023.

[3] K. Subashini and V. Narmatha, "Detecting phishing websites using recent techniques: A systematic literature review," ITM Web of Conferences, vol. 57, 2023.

[4] D. Kalla and S. Kuraku, "Phishing website URLs detection using NLP and machine learning techniques," Dec. 18, 2023.

[5] R. Jayaraj, A. Pushpalatha, K. Sangeetha, T. Kamaleshwar, S. Udhaya Shree, and D. Damodaran, "Intrusion detection based on phishing detection with machine learning," Dec. 21, 2023.

[6] S. Kapan and E. S. Gunal, "Improved phishing attack detection with machine learning: A comprehensive evaluation of classifiers and features," Dec. 15, 2023.

https://github.com/sibelkapan/phishing_dataset

[7] A. H. Aljammal, S. Taamneh, A. Qawasmeh, and H. Bani Salameh, "Machine learning based phishing attacks detection using multiple datasets," Int. J. Interactive Mobile Technologies, vol. 17, no. 5, 2023.

[8] A. A. Albishri and M. M. Dessouky, "A comparative analysis of machine learning techniques for URL phishing detection," vol. 14, no. 6, pp. 18495–18501, 2024.

[9] K. S. Jishnu and B. Arthi, "Real-time phishing URL detection framework using knowledge distilled ELECTRA," vol. 65, no. 4, pp. 1621–1639, 2024.

[10] M. A. Tamal, M. K. Islam, T. Bhuiyan, and A. Sattar, "Dataset of suspicious phishing URL

detection," Data Report, Mar. 6, 2024.

[11] A. U. Rehman, I. Imtiaz, S. Javaid, and M. Muslih, "Real-time phishing URL detection using machine learning," 2024.

[12] H. Ghalechyan, E. Israyelyan, A. Arakelyan, G. Hovhannisyan, and A. Davtyan, "Phishing URL detection with neural networks: An empirical study," 2024.

[13] F. Rashid, B. Doyle, S. C. Han, and S. Seneviratne, "Phishing URL detection generalisation using unsupervised domain adaptation," Apr. 6, 2024.

[14] "Leveraging machine learning algorithms for phishing website URL detection," Proc. National Conf. Emerging Computer Applications (NCECA), vol. 1.5, no. 1, 2024.

[15] S. K. Ahammad, S. D. Kale, G. D. Upadhye, S. D. Pande, E. V. Babu, A. V. Dhumane, and D. K. Bahadur, "Phishing URL detection using machine learning methods," 2024.

[16] Q. E. ul Haq, M. H. Faheem, and I. Ahmad, "Detecting phishing URLs based on a deep learning approach to prevent cyber-attacks," 2024.

[17] U. Zara, K. Ayyub, H. U. Khan, A. Daud, T. Alsahfi, and S. G. Ahmad, "Phishing website detection using deep learning models," Oct. 25, 2024.

[18] M. Abdolrazzagah-Nezhad and N. Langarib, "Phishing detection techniques: A review," vol. 9, no. 1, pp. 32–46, 2025.

[19] K. Barik, S. Misra, and R. Mohan, "Web-based phishing URL detection model using deep learning optimization techniques," published online Feb. 8, 2025.

[20] G. S. Nayak, B. Muniyal, and M. C. Belavagi, "Enhancing phishing detection: A machine learning approach with feature selection and deep learning models," Feb. 19, 2025.

[21] M. Murhej and G. Nallasivan, "Multimodal framework for phishing attack detection and mitigation through behavior analysis using EM-BERT and SPCA-based EAI-SC-LSTM," Jul. 8, 2025.

[22] H. Li, J. Yang, Y. Li, and K. Li, "Email phishing attack detection based on BERT transformer model," The Institute for Network Sciences and Cyberspace, Tsinghua University,

Beijing, China.

[23] M. A. A. Abdelrahman, H. M. A. Alblooshi, A. A. Ibrahim, F. Outay, and H. Malik, "WhatsPhish: AI powered WhatsApp phishing detector chatbot," Zayed University, Dubai, UAE.

[24] E. K. J, V. Surendhiran, S. Karuppusamy, and P. Vijayakumar, "An ensemble approach to phishing URL detection using supervised machine learning."

[25] M. V. Teja, N. Vyshnavi, N. Bhavni, V. Indrani, and A. Prashanthi, "Phishing URL detection using gradient boosting classifier."

[26] R. Alzubi, T. Bishtawi, and H. Kassem, "Improving web security through machine learning: A feature-based methodology for detecting phishing URLs."

[27] H. Fares, J. Kilani, F. Fagroud, H. Toumi, F. Lakrami, Y. Baddi, and N. Aknin, "Machine learning approach for email phishing detection."

[28] F. Salahdine, Z. El Mrabet, and N. Kaabouch, "Phishing attacks detection: A machine learning-based approach."

[29] J. L. Wilk-Jakubowski, L. Pawlik, G. Wilk-Jakubowski, and A. Sikora, "Machine learning and neural networks for phishing detection: A systematic review (2017–2024)."

[30] K. Barik, S. Misra, and R. Mohan, "Web-based phishing URL detection model using deep learning optimization techniques," published online Feb. 8, 2025.